

Retrieval-Augmented Generation (RAG) Systems

Page 1 of 2

Retrieval-Augmented Generation (RAG) is an architectural pattern that optimizes the output of a Large Language Model (LLM) by referencing an authoritative external knowledge base before generating a response. Standard LLMs are trained on vast datasets but remain frozen in time, leading to knowledge gaps, hallucinations, and inability to access private or dynamic business data. RAG addresses these critical limitations by introducing a retrieval step into the inference pipeline.

At its core, a RAG system works in three distinct phases: Ingestion, Retrieval, and Generation. During ingestion, documents are segmented into smaller semantic units called chunks, transformed into vector embeddings using representation models, and loaded into a vector database. At query time, the user's input is embedded, and a similarity search retrieves the most relevant document chunks. Finally, these chunks are injected into the LLM's prompt context, allowing the model to draft a grounded and factually accurate answer based strictly on the retrieved source material.

Retrieval-Augmented Generation (RAG) Systems

Page 2 of 2

The primary components of a modern RAG system include:

1. Document Ingestion Pipeline: Responsible for reading file formats (PDFs, HTML, Markdown), cleaning headers/footers, and splitting text. Chunking strategies typically use token-based sliding windows to maintain context across boundaries.
2. Vector Database: Acts as the storage engine for dense embeddings. It uses specialized index formats like HNSW (Hierarchical Navigable Small World) to enable sub-millisecond approximate nearest neighbor (ANN) searches.
3. Retriever: Pulls context from the vector database. High-quality systems use hybrid retrieval (combining dense semantic vector search with sparse lexical BM25 keyword matching) followed by a cross-encoder reranking model to filter out noisy results.
4. Generator: The LLM that synthesizes the retrieved information into a natural language response. Prompt engineering is applied to enforce citations (e.g., [filename, p.N]) and refuse answering if the context does not contain the answer.